CPSC 426 Assignment 3: Skeletal Animation

Due 11:59PM, March 15, 2026

1 Introduction

In this assignment you will implement a real-time skeletal animation system on the GPU. Starting from a rigged character mesh, you will build three core components:

1. **Forward Kinematics (FK)** — compute world-space bone transforms from local rotations.
2. **Linear Blend Skinning (LBS)** — deform the mesh according to the bone transforms.
3. **Inverse Kinematics (IK)** — automatically solve for joint angles that place an end-effector at a target position.

All GPU kernels are written with NVIDIA Warp [3], and the interactive UI is provided via Polyscope [1]. You will work from the provided starter code; your task is to implement the missing kernel and function logic in the `src/` folder. The starter code provides `TODO` comments as hints.

The assignment is designed so that each part can be verified visually before moving on:

- After **Part I**, the skeleton (bones and joints) appears and animates with the FK sliders.
- After **Part II**, the mesh deforms in response to FK controls.
- After **Part III**, IK gizmos drive the skeleton to reach target positions.

1.1 Getting the Code

Assignment code is hosted on the course's GitHub repository:
`https://github.students.cs.ubc.ca/CPSC426-2025W-T2/a3-release`
Please see the instructions in A2 for accessing the repository.

1.2 Template

1.2.1 Source Files

- `main.py` — Entry point and Polyscope UI. You do *not* need to modify this file.
- `forward_kinematics.py` — FK kernels.
  You will implement `compose_local_transforms` and `compute_world_transforms`. These kernels operate on Warp's compact `wp.transform` representation (7 floats: translation + quaternion). To obtain bind-pose world transforms, call `compose_local_transforms` with zero Euler angles and then `compute_world_transforms`.
- `skinning.py` — Skinning kernels.
  You will implement `compute_material_coords` and `compute_vertex_positions`.
- `ik_solver.py` — IK kernels and solver loop. You will implement `chain_fk_loss`, `step_gd`, and the optimization loop in `solve_ik`.
- `skeleton.py`, `mesh.py`, `cli.py`, `utils.py` — Helper files. You do *not* need to modify these.

1.2.2 Data

The `data/` directory contains:

- `skeleton_bind.json` — Defines the skeleton hierarchy: bone names, parent–child relationships, and bind-pose local 4×4 transformation matrices.
- `base_mesh.obj` — The character mesh (stored in Z-up meters; converted to Y-up centimeters on load).
- `vertex_weights.npz` — Per-vertex bone indices and weights for skinning (up to 4 bones per vertex, re-normalized).
- `lod3.fbx` — The original full-body rig from which the data was extracted. You can use this for reference in software like Blender.

All file loading is already implemented in `main.py` and `utils.py`. The original source data comes from the Momentum Human Rig (MHR) [2] assets, specifically the `lod3.fbx` fullbody rig, which we have converted to our custom text format for this assignment. You can import the `lod3.fbx` into Blender to compare your implementation to a ground truth.

1.3 Execution

```
python src/main.py              # uses preferred device (GPU if available)
python src/main.py --device cpu # force CPU execution
python src/main.py --IK         # enable IK controls and gizmos (Part III)
```

Please also read the `README.md` for setup instructions.

2 Pipeline Overview

Before diving into individual tasks, it helps to see how all the pieces connect. We will denote the world-space transform of bone k as $T_k = b_{w,k} * a_k$, where $b_{w,k}$ is the bind-to-world transform and a_k is the local animated transform produced from Euler angles.

The system operates in two phases:

Bind-time (runs once at startup). The skeleton's local bind transforms (translation + quaternion) are accumulated into world-space transforms using the quaternion FK kernels (**Part I**). These world-space `wp.transform` values are used to extract bone positions for the skeleton visualization and to compute inverse bind transforms required for material coordinates. If you specifically need bind-pose world transforms, call `compose_local_transforms` with a zero Euler-angle array and then call `compute_world_transforms` on the result.

Per-frame (runs on every UI interaction). User-controlled Euler angles are composed with bind-pose quaternions to form animated local rotations (**Part I**). These local rotations are accumulated along the hierarchy into world-space quaternion transforms (**Part I**). The LBS formula blends each vertex's material coordinates using the animated transforms and skinning weights (**Part II**). When IK is active (**Part III**), the solver runs *before* the per-frame FK pipeline, adjusting the Euler angles so the end-effector reaches the target.

3 Work to be Done (100 pts)

3.1 Part I: Forward Kinematics (30 pts)

In this part you implement the core FK pipeline using Warp's compact transform type `wp.transform` (7 floats: translation + quaternion). The two kernels live in `forward_kinematics.py` and are:

Task 1 (10 pts): Local Transform Composition (`compose_local_transforms`). Implement the `compose_local_transforms` kernel. This kernel composes each bone's animated local transform by applying the user-supplied Euler-angle offset to the bind-pose `wp.transform`. The input `bind_local_transforms` already contain both translation and rotation (translation + quaternion). For each bone k, convert its three Euler angles to a quaternion using XYZ rotation order (i.e., compose three axis-aligned rotations via `wp.quat_from_axis_angle` for X, Y, Z and multiply them as $q_x \cdot q_y \cdot q_z$), convert the result to a `wp.transform` a_k with 0 translation, and right-multiply the bind transform by it to form the composed local transform $b_{\text{local},k} * a_k$.

Task 2 (20 pts): World Transform Accumulation (`compute_world_transforms`). Implement the `compute_world_transforms` kernel. For each bone k we will compute a world transform T_k by combining the bind-to-world transform $b_{w,k}$ with the per-bone local transform a_k produced by Task 1:

$$T_k \;=\; b_{w,k} * a_k$$

('*' denotes `wp.transform_multiply`).

Where:

$$\begin{aligned} b_{w,k} &= T_{\text{parent\_of\_k}} * b_{\text{local, k}} \\ &= (b_{\text{local, root}} * a_{\text{root}}) * \cdots * (b_{\text{local, parent\_of\_k}} * a_{\text{parent\_of\_k}}) * b_{\text{local},k} \end{aligned}$$

Walking the parent chain effectively multiplies the parents' world transforms in the usual hierarchical order. The kernel therefore iterates up the chain accumulating T_k directly; the `wp.transform` API manages quaternion rotation and translation jointly.

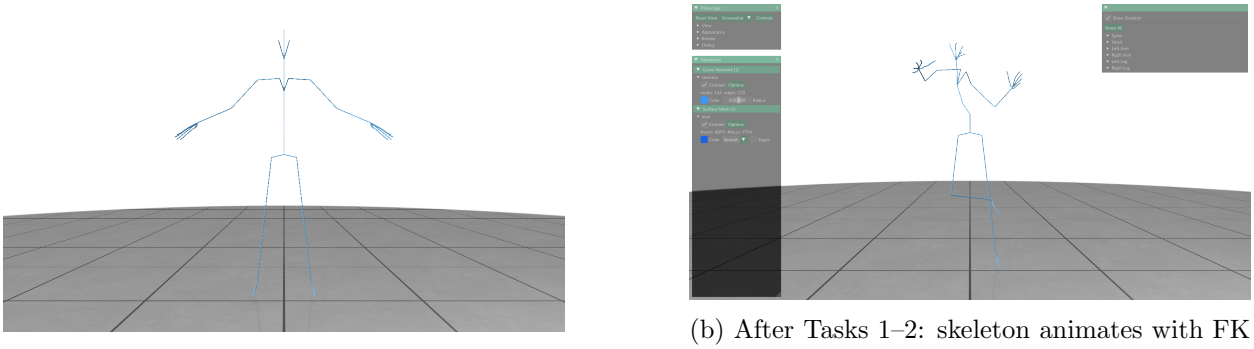
The bind-pose world transform $b_{w,k}$ computed when Euler angles are zero is also used elsewhere: its inverse $b_{w,k}^{-1}$ (the *inverse bind transform*) is needed in Part II to convert vertices into bone-local material coordinates.

Checkpoint: After completing Tasks 1–2, the skeleton should animate when you move the FK sliders. If the skeleton deforms correctly, your FK pipeline is working. The mesh will still not be visible until Part II.

3.2 Part II: Skinning (25 pts)

In this part you implement the skinning pipeline that deforms the mesh. Both kernels live in `skinning.py`.

Task 3 (10 pts): Material Coordinates. Complete the `compute_material_coords` kernel. Material coordinates express each mesh vertex in the local coordinate system of each influencing bone, computed once in the bind pose.



(a) After Tasks 1–2: skeleton in bind pose.

(b) After Tasks 1–2: skeleton animates with FK sliders.

Figure 1: Expected skeleton visualization after completing Part I.

For each vertex and each valid influencing bone (index $\neq -1$):

$$\mathbf{x}_{\text{material, k}} = \mathbf{b}_{\text{w, k}}^{-1} * \mathbf{x}_{\text{world,k}}$$

where $\mathbf{b}_{\text{w, k}}$ denotes the world-space bind-pose `wp.transform` for bone k, and $(\cdot)^{-1}$ indicates the inverse transform (obtainable via `wp.transform_inverse`). Compute the world-space bind transforms by calling `compose_local_transforms` with a zero Euler-angle array and then `compute_world_transforms`. For invalid bone entries (index $= -1$), set the material coordinate to zero.
These are pre-computed once at startup and reused every frame, avoiding per-frame inverse computation. Consult the Warp documentation for `wp.transform_inverse` and `wp.transform_point`.

Task 4 (15 pts): Linear Blend Skinning. Complete the `compute_vertex_positions` kernel. This deforms each vertex using the LBS formula:

$$\mathbf{x}'_{\text{world}} = \sum_j w_j \cdot \left(\mathbf{T}_j * \mathbf{x}_{\text{material},j}\right)$$

where w_j is the skinning weight for bone j, and $\mathbf{T}_j$ is the animated world-space transform of bone j (from Task 2). Skip invalid bone entries (index $= -1$) and bones with negligible weight[1].

Checkpoint: After completing Part II, the mesh should appear in the A-pose and deform correctly when you move the FK sliders. Both the skeleton and mesh should move together.

3.3 Part III: Inverse Kinematics (30 pts)

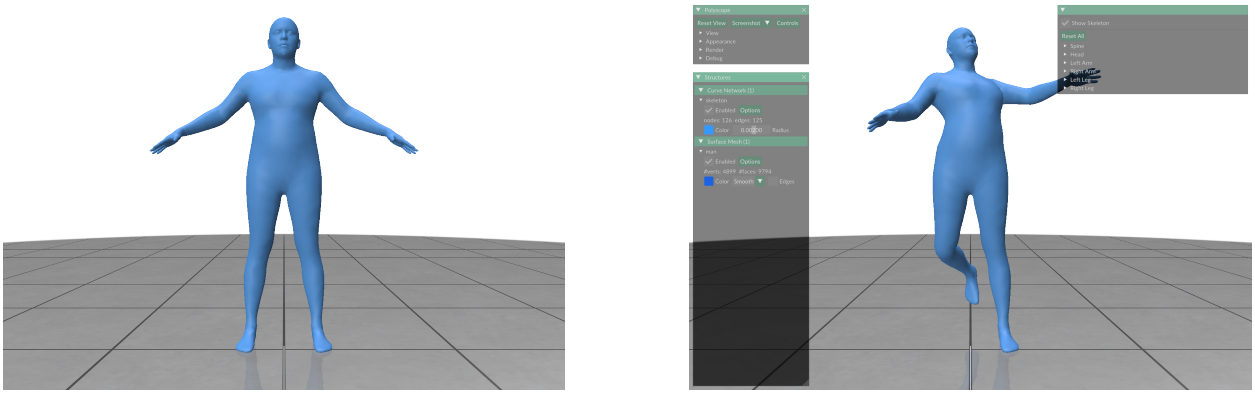
In this part you implement a gradient-descent IK solver using Warp's automatic differentiation. All code lives in `ik_solver.py`.

When testing this part, use CLI flag `--IK`; this enables the IK UI controls and gizmos in the Polyscope viewer.

Background. The IK solver finds joint angles $\boldsymbol{\theta}$ that minimize the squared distance between an end-effector position and a user-specified target. Rather than forming and inverting a Jacobian, we define a differentiable scalar loss and let Warp's `wp.Tape` compute $\partial\text{Loss}/\partial\boldsymbol{\theta}$ automatically.

The starter code provides four IK chains (left/right arm, left/right leg), each with 2 controlled joints:

[1]We are performing linear-blend skinning on a constant number of bones for each vertex instead of using every bone; this is a common practice to reduce computational cost in animating meshes.



(a) Mesh and skeleton in bind pose.

(b) FK sliders deform the mesh.

Figure 2: Expected visual results after completing Part II.

Chain	Controlled Joints	End Effector
Left Arm	`l_uparm`, `l_lowarm`	`l_wrist`
Right Arm	`r_uparm`, `r_lowarm`	`r_wrist`
Left Leg	`l_upleg`, `l_lowleg`	`l_foot`
Right Leg	`r_upleg`, `r_lowleg`	`r_foot`

A helper function `_build_chain_path` traces the bone hierarchy from the root to the end-effector and produces an `euler_map` that identifies which bones in the chain are controlled joints.

Task 5 (15 pts): Differentiable FK Loss. Complete the `chain_fk_loss` kernel. This kernel performs forward kinematics for a single kinematic chain and outputs the squared distance between the end-effector position and the target:

$$\text{Loss} = \|\mathbf{p}_{\text{ee}} - \mathbf{p}_{\text{target}}\|^2$$

The kernel walks the chain from root to end-effector (using `chain_path` and `chain_length`), accumulating world-space quaternion and translation—the same FK logic as Task 2. For *controlled* joints (`euler_map[i] >= 0`), you must recompute the local quaternion from the `euler_angles` array using the same Euler-to-quaternion conversion as Task 1. This is necessary so that Warp's autodiff can trace gradients from the Euler angles to the loss. For uncontrolled joints, use the pre-computed `local_transforms`.

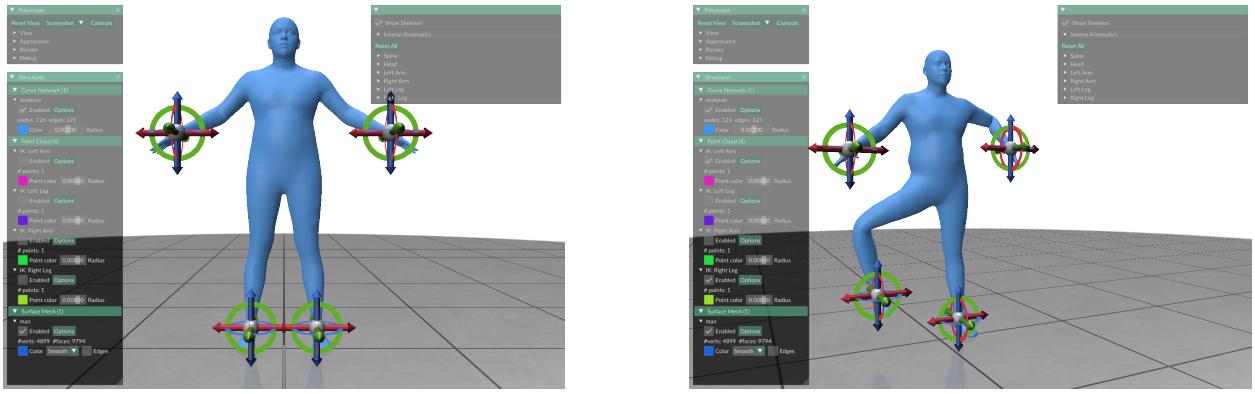
Important: Warp's autodiff requires *static* loop bounds—you cannot use a `while` loop here. The constant `MAX_CHAIN_DEPTH` and the `chain_length` array are provided for this purpose. Use a `for i in range(MAX_CHAIN_DEPTH)` loop and guard the body with `if i < n`.

Task 6 (15 pts): Gradient Descent Update and Solver Loop. Implement two things:

(a) The `step_gd` kernel: a per-element gradient descent update $\theta \leftarrow \theta - \eta \cdot \nabla\theta$.

(b) The optimization loop inside `solve_ik`. The chain path construction and Warp array setup are provided; you implement the iterative part. Each iteration should record a differentiable forward pass, compute gradients via `wp.Tape`, update the joint angles, and enforce joint limits. Consult the Warp documentation on `wp.Tape` and the `TODO` comments in the starter code.

Checkpoint: After completing Part III, enable an IK chain in the UI (e.g., "Enable Left Arm IK"), then drag the gizmo. The arm (or leg) should follow the target in real time.



(a) IK enabled: gizmo at end-effector.

(b) IK solved: arm follows the target.

Figure 3: Expected IK behavior after completing Part III.

3.4 Part IV: Feature Extension (extra 15 pts)

Extend the assignment with a feature of your choice. Some ideas:

- **Alternative IK Solver:** Implement a different IK algorithm such as FABRIK or CCD and compare it to the gradient-based method.
- **Animation System:** Create a keyframe animation system. Allow the user to save poses (joint angles) at specified times and interpolate between them (e.g., SLERP for quaternions, linear for translations).
- **Additional IK Chains:** Extend the IK system to support more chains (e.g., spine, head tracking) or add constraints such as pole vectors for controlling elbow/knee direction.
- **Dual Quaternion Skinning:** Implement dual quaternion skinning as an alternative to LBS and compare the two methods, particularly around twist deformations.

4 Summary of Tasks

#	File	Kernel / Function	Part	Pts
1	`forward_kinematics.py`	`compose_local_transforms`	I (FK)	10
2	`forward_kinematics.py`	`compute_world_transforms`	I (FK)	20
3	`skinning.py`	`compute_material_coords`	II (Skinning)	10
4	`skinning.py`	`compute_vertex_positions`	II (Skinning)	15
5	`ik_solver.py`	`chain_fk_loss`	III (IK)	15
6	`ik_solver.py`	`step_gd` + `solve_ik` loop	III (IK)	15
7	`Feature Extension`		IV	15
			Total	**100**

Recommended order: Tasks 1 → 2 (verify with skeleton) → 3 → 4 (verify with mesh) → 5 → 6 (verify with IK gizmos).

5 Tips

- Pay close attention to the order of operations when accumulating transforms along a parent chain.
- Quaternion multiplication is non-commutative: $\mathbf{q}_a \cdot \mathbf{q}_b \neq \mathbf{q}_b \cdot \mathbf{q}_a$.
- Use the skeleton visualization to debug FK before moving on to skinning.
- The IK loss kernel reuses the same FK math you already implemented in Part I.

6 Submission Instructions

6.1 Directory Structure

Your submission should maintain a `src/` folder for the basic requirements, so that in F2F grading we can easily assign marks for the first two parts. For feature extensions, please create a `src2/` folder including your changes. If your feature extension requires more complex separation, you can tweak this. We only ask that you do not duplicate excessively (e.g., do not copy `data/` twice).

Please add your name, student number, and CWL username in the existing `README.md` file. You should modify the first line's template and replace it with your information. Also, you can include any information you would like to pass on to the marker below that (otherwise leave N/A is fine).

6.2 Submission Methods

Please compress everything under the root directory of your assignment into an `a3.zip` and submit it on Canvas. You can make multiple submissions, but we will grade only the last one.

7 Grading

7.1 Face-to-face (F2F) Grading

To get a grade for the assignment, you must meet face-to-face with a TA in a ∼10-min slot on Zoom to demonstrate that you understand how your program works. Grading slots and detailed instructions on how to sign up and conduct F2F grading will be announced on Canvas and on Piazza.

While the exact process and questions may vary, you can generally expect the following during the meeting: The TA will (1) look at and/or run your code to inspect its correctness; (2) ask you to explain parts of your code, **including Python syntax and library features you use**; (3) ask you understanding-based questions about the assignment and logic. You are expected to answer them in a reasonable timeframe (based on difficulty).

The questions are intended to be mostly based on the code we ask you to implement, so forward kinematics, quaternion transforms, linear blend skinning, inverse kinematics, and Warp concepts that you will have seen in lecture and while completing the assignment. We hope that the questions are more conversational than robotic; not everything we say is a test to deduct points from you.

In most cases, any awkwardness or nervousness is just noise, so feel free to take pauses and ask for clarifications if anything confuses you. We may ask questions which ask deeper questions of "why", and from that perspective it is okay to not immediately have an answer. We want to see your thought process in figuring out one.

7.2 Point Allocation

All questions before Feature Extension have a total of 85 points. The points are warranted based on

- The functional correctness of your program, i.e., how visually close your results are to expected results;
- The algorithmic correctness of your program, e.g., correctly implementing the FK kernels, Euler-to-quaternion conversion, linear blend skinning, and IK solver;
- Your demonstration of understanding of the code through your answers to TAs' questions during face-to-face (F2F) grading.

The feature extension grade is based on creativity and sophistication of your submission. We warrant partial marks on four scales:

- 15: Outstandingly creative and sophisticated feature extension, e.g. the suggestions provided in Sec. 3.4. But note that only a very small number of extensions may get this score, and we cannot guarantee that if you implement one of the suggestions as is then you will definitely get full marks;
- 10: Average creativity and sophistication, e.g. adding pole vector constraints, implementing keyframe interpolation with SLERP;
- 5: Basic creativity and sophistication, e.g. uses the existing APIs to combine into a novel result;
- 0: Trivial or no feature extension, e.g. only changes CLI parameters, making purely cosmetic tweaks without new logic, or adding an extension that does not work reliably / cannot be explained during grading.

7.3 Collaboration Policy

Submissions must be individual, but you are encouraged to discuss the assignment with your classmates, and you are allowed to use generative AI tools, e.g. ChatGPT, Claude to help with constructing your solution.

In the beginning lines of the `README.md` file, include names of the people you discussed the assignment with, websites you acquired information from, and if you used generative AI tools, include the names of the tools you used.

You must understand your code and be able to explain it during F2F grading. If you cannot demonstrate at least a basic understanding of your code during face-to-face grading, you may receive a zero for the assignment—even if your code runs perfectly. Hiring a tutor to write your

assignment, or copy-pasting others' code without proper referencing in `README.md` are considered as plagiarism, and will be reported to the dean's office.

7.4 Penalties

Aside from penalties from incorrect solution or plagiarism, we may apply the following penalties to each assignment:

Late penalty. You are entitled up to three grace (calendar) days in total throughout the term. No penalties would be applied for using them. However once you have used up the grace days, a deduction of 10 points would be applied to each extra late day. Note that

1. The three grace days are given for all assignments, **not per assignment**, so please use them wisely;
2. We check the time of your last submission to determine if you are late or not.

No-show penalty. Please sign up for a grading slot on the provided sign-up spreadsheet (link will be posted later on Piazza) before the submission deadline of the assignment, and show up to your slot on time. A 10-point deduction would be applied to each of the following circumstances:

1. Not signing up for a grading slot before the sign-up period closes. Unless otherwise stated, the period closes at the same time as the submission deadline.
2. Not showing up at your grading slot.

If none of the provided slots work for you, or if you have already missed your slot, follow instructions outlined in the given Piazza post for F2F grading to get graded after the grading period ends. Also, please note the following:

1. You will need to explain to your TA why you're getting graded late, and may be asked to present documents to justify your hardship. The TA may remove the no-show penalty as long as he/she deems the justification to be reasonable.
2. In the past some students reported that their names disappeared mysteriously due to technical glitches, and the spreadsheet's edit history has no trace of it. So double check that your name is on the sign-up sheet after you sign up by refreshing the page.
3. Please do not try to modify other students' times or act in bad faith. Again, the sign-up sheet has history.

References

[1] Nicholas Sharp and others. *Polyscope*, 2019. `https://www.polyscope.run`

[2] Aaron Ferguson et al. "MHR: Momentum Human Rig", 2025. arXiv:2511.15586. `https://github.com/facebookresearch/MHR`

[3] NVIDIA. *Warp Documentation*. `https://nvidia.github.io/warp/`